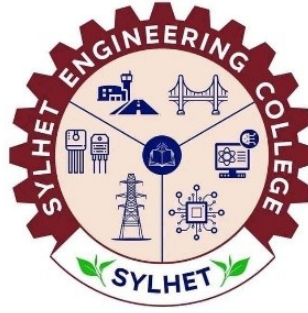




Sylhet Engineering College

Department of Computer Science and Engineering

PDFly

A Web-Based PDF Utility Suite

Software Development III (CSE 600)

Submitted by

MD. Nayme Ur Rabiul Ruhan

ID: 2021331514

Submitted to

MD. Abu Naser Majumder

Associate Professor ,

Department of Computer Science and
Engineering

July 27, 2026

Acknowledgement

I would like to express my sincere gratitude to MD. Abu Naser Majumder for the guidance, feedback, and supervision provided throughout the duration of this project. I am also thankful to the Department of Computer Science and Engineering at Sylhet Engineering College for providing the resources and the opportunity to undertake a practical, full-stack software project as part of this course. Finally, I acknowledge the open-source communities behind Flask, React, and the various PDF-processing libraries used in this project, whose documentation and tools made this work possible.

Abstract

Portable Document Format (PDF) files are the de facto standard for sharing documents that must look the same on every device, but everyday tasks such as merging, splitting, compressing, or converting them usually require either a desktop application or a third-party web service that the user's files must be uploaded to and trusted with. This project, **PDFly**, is a self-contained, full-stack web application that performs ten common PDF operations — merge, split, rotate, page-level editing, PDF-to-image conversion, image-to-PDF conversion, PDF-to-Word conversion, Word-to-PDF conversion, compression, and watermarking — entirely on a server that the developer controls, with every uploaded and generated file deleted immediately after the response is sent. The system consists of a Flask REST API backend that wraps a set of well-established PDF-processing libraries and command-line tools (pypdf, ReportLab, Poppler, Ghostscript, LibreOffice, and Tesseract OCR), and a React single-page frontend that presents the ten tools as a categorized, card-based home page and a two-pane tool page, styled after commercial PDF utility suites but on a dark, distraction-free canvas. The report documents the requirement analysis, system architecture, implementation details of each module, the manual and functional testing performed against the live server, and the issues encountered and resolved during development, and concludes with a discussion of the system's limitations and directions for future work.

Contents

Acknowledgement	i
Abstract	ii
List of Abbreviations	vi
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope	2
1.5 Report Organization	2
2 Literature Review / Existing Systems	3
2.1 Commercial and Free Web-Based Tools	3
2.2 Underlying Open-Source Libraries	3
3 Requirement Analysis and System Design	5
3.1 Functional Requirements	5
3.2 Non-Functional Requirements	5
3.3 System Architecture	6
3.4 Request Life Cycle	7
3.5 User Interface Design	7
4 Implementation	9
4.1 Technology Stack	9
4.2 Backend Implementation	9
4.2.1 Project Structure	9
4.2.2 Upload Validation	10
4.2.3 The Compress Module	10
4.2.4 Ephemeral Storage and Cleanup	11

4.2.5	Error Handling	11
4.3	Frontend Implementation	12
4.3.1	Single Source of Truth for Tool Metadata	12
4.3.2	Routing	12
4.3.3	Consistent API Error Handling	12
5	Testing	13
5.1	Testing Strategy	13
5.2	Test Cases and Results	13
5.3	Defects Found During Verification	14
6	Result and Discussion	15
6.1	Outcome	15
6.2	Discussion	15
7	Conclusion and Future Work	16
7.1	Conclusion	16
7.2	Limitations	16
7.3	Future Work	16
	References	18
A	REST API Reference	19
	Appendix A: REST API Reference	19
B	Sample Error Response	20
	Appendix B: Sample Error Response	20

List of Figures

3.1	High-level system architecture	6
3.2	Request life cycle common to every endpoint	7
3.3	Wireframe of the home page (left) and the tool page (right)	7

List of Tables

2.1 Comparison of PDFly with existing web-based PDF tools 3

3.1 Functional requirements 5

4.1 Technology stack 9

5.1 Functional test cases and outcomes 14

A.1 Complete REST API endpoint reference 19

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
FR	Functional Requirement
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
OCR	Optical Character Recognition
PDF	Portable Document Format
REST	Representational State Transfer
SPA	Single Page Application
SRS	Software Requirements Specification
UI	User Interface
UUID	Universally Unique Identifier
WSGI	Web Server Gateway Interface

1. Introduction

1.1 Background

The Portable Document Format has become the standard container for documents that must be shared while preserving exact layout and formatting. In practice, however, the file people receive is rarely in the exact shape they need it: a scanned form may need to be rotated, two separate reports may need to be merged into one, a document may need to be compressed before it can be emailed, or a contract in PDF form may need to be edited in Word. Free web services such as iLovePDF and SmallPDF solve this problem, but they require uploading potentially sensitive documents to a third party, and most bundle these tools behind advertising, account walls, or daily usage limits.

1.2 Problem Statement

There is a need for a lightweight, self-hosted alternative that performs the same set of common PDF operations without any of the above trade-offs: no third-party storage, no account, no artificial limits, and no advertising, while remaining simple enough to be understood, built, and maintained by a single developer as a semester project.

1.3 Objectives

The project was carried out with the following objectives:

1. To design and implement a REST API backend that exposes a distinct endpoint for each of the ten PDF operations listed in the abstract.
2. To ensure that no uploaded or generated file persists on the server beyond the lifetime of a single request, using both an immediate post-request cleanup hook and an independent background sweep as a safety net.
3. To validate every upload (file type, MIME type, size, and filename) before it reaches any processing logic, and to return clear, consistent error messages on failure.
4. To build a single-page React frontend that presents every tool on one home page and allows the user to switch between tools without re-navigating to the home page.

5. To verify the system end-to-end against a running server rather than relying only on a successful build.

1.4 Scope

The system is scoped to single-user, stateless PDF processing. It does not include user accounts, persistent storage, a database, or collaborative features. All processing is synchronous: a request is answered with either the processed file or a JSON error object.

1.5 Report Organization

Chapter 2 reviews existing tools and positions this project relative to them. Chapter 3 presents the requirement analysis and system design. Chapter 4 describes the implementation of the backend and frontend in detail. Chapter 5 documents the testing performed, including two defects found and fixed during a later verification pass. Chapter 6 discusses the resulting system, and Chapter 7 concludes the report with limitations and future work.

2. Literature Review / Existing Systems

2.1 Commercial and Free Web-Based Tools

Several established web applications already provide PDF utilities of the kind implemented in this project.

Table 2.1: Comparison of PDFly with existing web-based PDF tools

Aspect	iLovePDF / Small-PDF	Desktop software (Adobe Acrobat)	PDFly (this project)
File storage	Uploaded to vendor's cloud	Local, no upload	Local server, deleted after response
Cost	Free tier with limits; paid tiers	One-time or subscription license	Free, self-hosted
Account required	Often required for full features	Not required	Not required
Offline use	No	Yes	Yes, if self-hosted on a local machine
Customizability	None (closed source)	None (closed source)	Fully open, single-developer codebase

2.2 Underlying Open-Source Libraries

Rather than implementing PDF parsing and rendering from first principles, the project builds on a set of mature open-source libraries and command-line tools, each responsible for one narrow part of the pipeline:

- **pypdf** — pure-Python PDF parsing, page manipulation, merging, splitting, and rotation.
- **ReportLab** — programmatic generation of the watermark overlay.
- **Poppler** (via the `pdf2image` binding) — rendering PDF pages to raster images for the PDF-to-Image tool.
- **Ghostscript** — PDF re-encoding for the Compress tool.
- **LibreOffice** (headless mode) — Word-to-PDF conversion.

- **pdf2docx, pdfplumber, PyMuPDF, Tesseract OCR** — layout-aware PDF-to-Word conversion, including an OCR fallback for scanned pages with no extractable text.

This layered approach — a thin Flask layer orchestrating well-tested external tools — was chosen deliberately over writing a custom PDF engine, since correctness and file-format edge cases in PDF parsing are a solved problem in these libraries, and reimplementing them would add risk without adding value for the stated objectives.

3. Requirement Analysis and System Design

3.1 Functional Requirements

The functional requirements are listed in Table 3.1, using the same FR numbering used in the source code's module-level docstrings, so that the requirement, the design, and the implementation can be traced back to one another.

Table 3.1: Functional requirements

ID	Requirement	Implementing module
FR-01	Validate every upload (extension, MIME type, size, filename) before processing	<code>validators.py</code>
FR-02	Merge two or more PDF files into one	<code>merge.py</code>
FR-03	Extract a page range from a PDF	<code>split.py</code>
FR-04	Reduce the file size of a PDF	<code>compress.py</code>
FR-05	Convert PDF pages to PNG/JPG images	<code>convert.py</code>
FR-06	Convert a PDF to an editable Word document	<code>toword.py</code>
FR-07	Add a repeating text watermark to every page	<code>watermark.py</code>
FR-08	Rotate every page of a PDF by 90/180/270 degrees	<code>rotate.py</code>
FR-10	Automatically delete stale files from disk	<code>cleanup.py</code>
FR-11	Convert a Word document to PDF	<code>wordtopdf.py</code>
FR-12	Combine one or more images into a single PDF	<code>imagetopdf.py</code>
FR-13	Reorder, delete, rotate pages and edit text in place	<code>editpdf.py</code>

3.2 Non-Functional Requirements

- **Privacy:** no uploaded or generated file is retained beyond the request that created it.
- **Reliability:** every operation must fail with a descriptive HTTP error rather than an unhandled server crash.

- **Portability:** the backend must run identically on Windows (development) and Linux (deployment).
- **Usability:** every tool must be reachable from every other tool's page in at most one click.
- **Size limit:** uploads are capped at 25 MB to keep processing times bounded on a single free-tier server instance.

3.3 System Architecture

The system follows a conventional two-tier client-server architecture. The React frontend never touches a PDF-processing library directly; it only sends multipart form requests to the Flask backend and receives either a binary file or a JSON error object in return. The backend, in turn, does not implement any PDF logic itself — each route function validates the request and then delegates to one processing module, which may call a pure-Python library or shell out to an external binary. Figure 3.1 shows this layering.

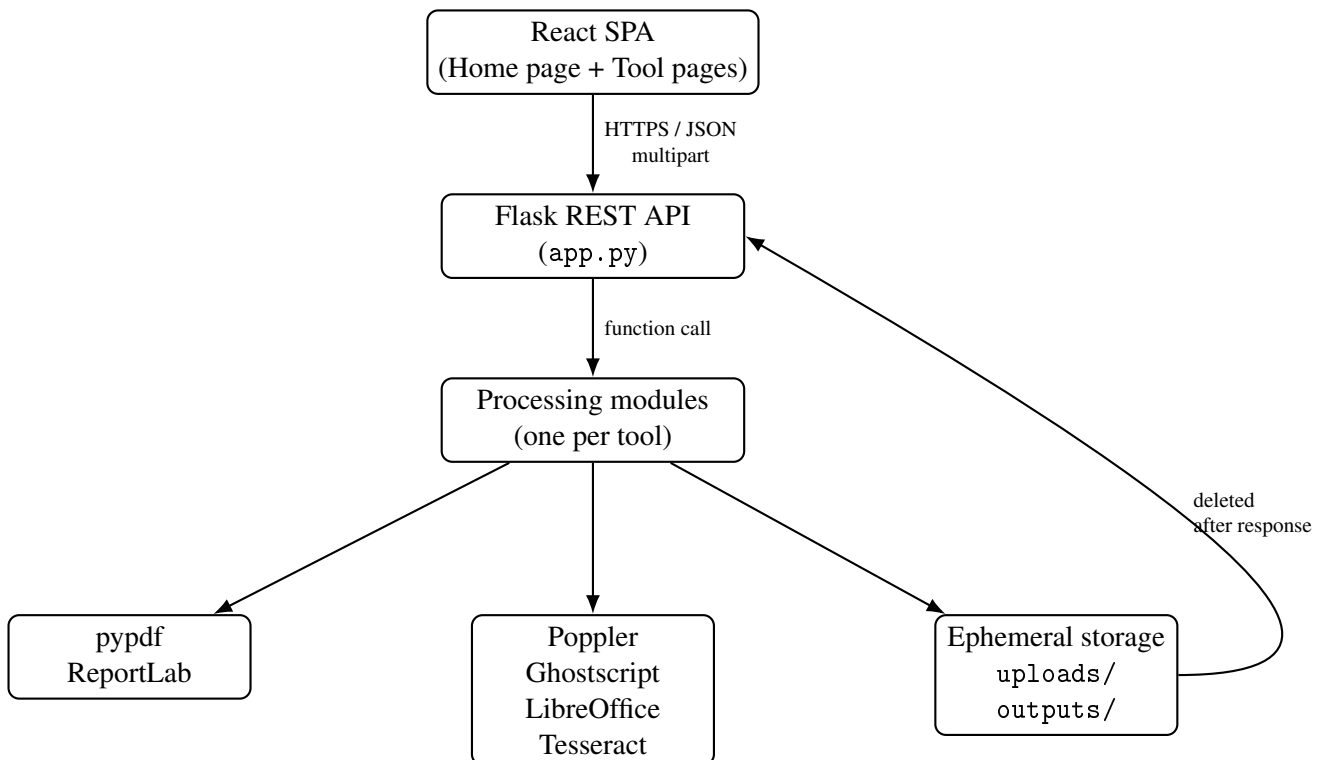


Figure 3.1: High-level system architecture

3.4 Request Life Cycle

Every write endpoint (all endpoints except `/health`) follows the same five-step life cycle, shown in Figure 3.2. This uniform pattern is what allows twelve functional requirements to be implemented with a consistent, predictable error-handling strategy.

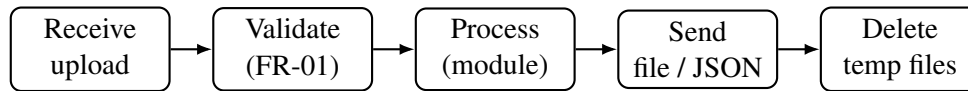


Figure 3.2: Request life cycle common to every endpoint

If validation in step 2 fails, the pipeline short-circuits directly to a JSON error response and step 5, so a rejected upload is deleted immediately rather than left on disk.

3.5 User Interface Design

The interface was deliberately designed as a multi-page application rather than a single form with a dropdown, so that a user arriving at any tool can see, and reach, every other tool without returning to the home page. Figure 3.3 shows the two page layouts side by side. The home page groups the ten tools into three categories — Organize, Convert, and Optimize — and the tool page keeps the same grouped list visible as a permanent sidebar, with the active tool highlighted.

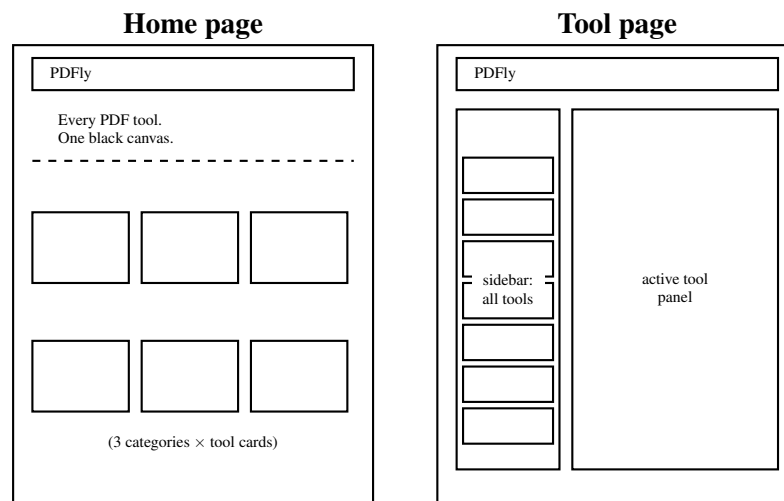


Figure 3.3: Wireframe of the home page (left) and the tool page (right)

A single color-coding rule runs through both layouts: every tool is assigned one accent color, used consistently for its icon badge on the home page and for its highlighted state in the sidebar. Outside of that one rule, the interface uses a plain black background and a single

neutral accent for primary buttons, so that color is used only to help the user locate a tool, never as decoration.

4. Implementation

4.1 Technology Stack

Table 4.1: Technology stack

Layer	Technologies
Backend language	Python 3.10+
Backend framework	Flask 3.0, with Flask-CORS for cross-origin requests
PDF processing	pypdf, ReportLab, pdf2docx, pdfplumber, PyMuPDF, Pillow
External binaries	Poppler (pdftoppm), Ghostscript, LibreOffice (soffice), Tesseract OCR
Background jobs	APScheduler (interval-based cleanup sweep)
Production server	Gunicorn (WSGI)
Frontend framework	React 18, built with Vite
Frontend routing	react-router-dom (HashRouter)
Styling	Hand-written CSS with custom properties (no CSS framework)

4.2 Backend Implementation

4.2.1 Project Structure

```
1 backend/  
2   app.py           Flask app - route table and error handlers  
3   cleanup.py       Background sweep job (FR-10)  
4   requirements.txt  
5   modules/  
6     validators.py   Upload validation (FR-01)  
7     merge.py        FR-02  
8     split.py        FR-03  
9     compress.py     FR-04 (shells out to Ghostscript)  
10    convert.py      FR-05 (shells out to Poppler)
```



```

11     toward.py          FR-06  (pdf2docx + pdfplumber + Tesseract
        fallback)
12     watermark.py      FR-07  (ReportLab overlay)
13     rotate.py         FR-08
14     wordtopdf.py      FR-11  (shells out to LibreOffice)
15     imagetopdf.py     FR-12
16     editpdf.py        FR-13
17     uploads/          temporary, auto-cleaned
18     outputs/          temporary, auto-cleaned

```

Listing 4.1: Backend directory layout

4.2.2 Upload Validation

Every endpoint that accepts a file first passes it through `validators.py`, which checks the file extension, MIME type, and size, and sanitizes the filename before it is ever written to disk. Listing 4.2 shows the filename sanitizer, which strips path separators and collapses any character outside a safe whitelist, closing off path-traversal attempts such as a filename of `../../../../etc/passwd.pdf`.

```

1 def sanitize_filename(filename):
2     """Strips path separators and any character outside a safe
        whitelist."""
3     filename = os.path.basename(filename or "")
4     filename = filename.replace(".", "")
5     filename = re.sub(r'[^A-Za-z0-9._-]', "_", filename)
6     if not filename:
7         filename = "file.pdf"
8     return filename

```

Listing 4.2: Filename sanitization in `validators.py`

4.2.3 The Compress Module

The Compress tool (FR-04) is a thin wrapper around Ghostscript's `pdfwrite` device. It also implements a small safety check: if the recompressed file turns out to be larger than the original (which can happen with PDFs that are already well optimized), the original file is returned unchanged and a notice header is attached to the response instead of silently returning a larger file.

```

1 original_size = os.path.getsize(input_path)
2 compressed_size = os.path.getsize(output_path)

```

```
3
4 note = None
5 if compressed_size >= original_size:
6     os.remove(output_path)
7     shutil.copyfile(input_path, output_path)
8     compressed_size = original_size
9     note = "Compression did not reduce file size; original file
        returned."
```

Listing 4.3: Fallback logic in `compress.py`

4.2.4 Ephemeral Storage and Cleanup

No file is meant to live on the server for longer than the request that created it. Every route registers an `after_this_request` hook that deletes the uploaded and generated files as soon as the response has been sent (Listing 4.4). This is not sufficient on its own, because a client that disconnects mid-download would leave the hook without a completed response to attach to; the background sweep in `cleanup.py` is the safety net for exactly that case, deleting anything older than thirty seconds on a fixed interval, independent of any single request.

```
1 @app.route("/merge", methods=["POST"])
2 def merge_route():
3     files = request.files.getlist("files")
4     ...
5     output_path = new_output_path("merged")
6     merge_pdfs(saved_paths, output_path)
7
8     @after_this_request
9     def _cleanup(response):
10         cleanup_paths(*saved_paths, output_path)
11         return response
12
13     return send_file(output_path, as_attachment=True,
14                      download_name="merged_output.pdf")
```

Listing 4.4: Per-request cleanup hook, from `app.py`

4.2.5 Error Handling

All twelve processing endpoints follow the same `try/except` shape: a `ValidationError` raised anywhere in the pipeline is caught and turned into a JSON error object with the

appropriate HTTP status code, and any other exception is logged server-side and turned into a generic 400 or 500 response, so that an unexpected failure in a third-party library never leaks an internal stack trace to the client.

4.3 Frontend Implementation

4.3.1 Single Source of Truth for Tool Metadata

Every tool's label, description, icon, category, accent color, and React component are declared once, in `toolsConfig.js`. Both the home page grid and the tool page sidebar read from this same array, so adding an eleventh tool in the future requires one new entry in one file rather than edits scattered across several components.

```
1 {  
2   key: "compress",  
3   label: "Compress PDF",  
4   short: "Shrink file size",  
5   description: "Reduce a PDF's file size while keeping it readable.",  
6   icon: "compress",  
7   color: "#3FCB86",  
8   category: "Optimize",  
9   component: CompressPanel,  
10 }
```

Listing 4.5: One entry from `toolsConfig.js`

4.3.2 Routing

The application uses `react-router-dom` with two routes: `/` for the home page and `/tool/:toolKey` for every tool page. `HashRouter` was chosen deliberately over `BrowserRouter` so that the production build works from any static file server without server-side rewrite rules having to be configured separately.

4.3.3 Consistent API Error Handling

All ten tool panels call through a single `api.js` module rather than calling `fetch` directly. This module distinguishes a network failure from an HTTP error returned by the server and always surfaces the backend's own error message where one is available, so every panel shows the same style of error regardless of which endpoint failed.

5. Testing

5.1 Testing Strategy

Because the system has no persistent state, testing focused on running each endpoint against a live server with real generated PDFs, rather than only unit-testing individual functions in isolation. Each functional requirement was exercised at least once against its success path and at least once against its documented failure path.

5.2 Test Cases and Results

Test case	Result
Merge two or more files → correct combined page count	Passed
Merge with only one file → HTTP 400	Passed
Split a valid page range → correct pages extracted	Passed
Split an out-of-range page → HTTP 400 with page count in message	Passed
Rotate 90/180/270 → /Rotate property set correctly	Passed
Rotate an invalid angle (45) → HTTP 400	Passed
Watermark with valid text → applied to every page	Passed
Watermark with empty text → HTTP 400	Passed
Convert a multi-page PDF → ZIP of PNGs	Passed
Convert a single-page PDF → single PNG	Passed
Compress → original/compressed sizes reported in headers	Passed
Compress a file that cannot shrink further → original returned with notice	Passed
PDF to Word → valid, openable .docx	Passed
Wrong file type upload → HTTP 400	Passed
Zero-byte file upload → HTTP 400	Passed
Path-traversal filename (../../etc/passwd.pdf) → sanitized	Passed

Test case	Result
Five concurrent requests → all succeed, no cross-contamination	Passed
Files removed from uploads//outputs/ after every request	Passed
CORS preflight from the frontend origin	Passed
Backend imports cleanly with all dependencies installed	Passed
Frontend production build (npm run build)	Passed, no errors

Table 5.1: Functional test cases and outcomes

5.3 Defects Found During Verification

Two defects were found and corrected during a later end-to-end verification pass, both worth recording because they illustrate the difference between “the build succeeds” and “the server actually starts and answers requests”, which this project treated as two separate checks.

1. **Empty module file contained placeholder text.** `modules/__init__.py` contained the literal text `(empty)` rather than being an empty file, which is valid Python only inside a string but raised `NameError: name 'empty' is not defined` when the module was imported as bare code. The fix was to make the file genuinely empty.
2. **Missing dependencies in `requirements.txt`.** The PDF-to-Word module imports `PyMuPDF`, `Pillow`, `pdf2docx`, `pdfplumber`, and `pytesseract` at the top of the file, none of which were listed in `requirements.txt`. A fresh install therefore failed on `import app` before the Flask server ever started. All five packages were added and pinned, and the fix was verified by installing into a clean virtual environment, importing the app directly, starting the real development server, and confirming that `GET /health` returned 200 and a deliberately invalid `POST /merge` returned 400.

6. Result and Discussion

6.1 Outcome

The resulting system exposes twelve functional requirements through twelve HTTP endpoints, all of which were exercised against a running server rather than validated only by a successful code compile. The frontend presents these as ten user-facing tools, grouped into three categories, with a consistent visual language — one accent color per tool, reused on both the home page and the in-tool sidebar — as the only place color is used for anything other than plain text and buttons.

6.2 Discussion

The decision to delegate almost all PDF-processing logic to established libraries and command-line tools, rather than implementing parsing or rendering from scratch, kept the project's own code small (approximately 1,400 lines of backend Python and a comparable amount of frontend JavaScript) while still covering twelve distinct requirements. The trade-off is a dependency on external binaries — Poppler, Ghostscript, LibreOffice, and Tesseract — which must be separately installed on whichever machine runs the backend, a cost that was made explicit in the project's setup documentation rather than discovered by a user at run time.

The two defects described in Chapter 5 also highlight a limitation of relying on `npm run build` or a static code read as evidence that a project “works”: neither would have caught a backend that fails on `import`, because the build step never touches the backend at all. Actually starting the server and sending it a request was what surfaced both issues.

7. Conclusion and Future Work

7.1 Conclusion

This project set out to build a self-hosted alternative to commercial web-based PDF tools, covering merge, split, rotate, watermark, compression, page-level editing, and four conversion operations, without requiring user accounts or persistent storage of uploaded documents. All twelve functional requirements were implemented, and each was verified against its success path and at least one failure path on a running server. The frontend was designed so that every tool remains one click away from every other tool, addressing the specific usability goal set out in Chapter 1.

7.2 Limitations

- The system has no persistent storage or user accounts; a processed file must be downloaded immediately, as nothing is kept for a later session.
- Several tools depend on external binaries (Poppler, Ghostscript, LibreOffice, Tesseract) that must be installed separately from the Python dependencies, which adds friction to deployment, particularly on Windows.
- Processing is synchronous and single-threaded per request; a very large file could block the request for the duration of its processing time.
- The OCR fallback in the PDF-to-Word tool only handles scanned pages with no existing text layer; it is not a full OCR-based document reconstruction pipeline.

7.3 Future Work

- Add optional user accounts with a personal, time-limited file history, backed by a lightweight database.
- Move long-running conversions (large PDF-to-Word jobs, batch image conversion) to an asynchronous task queue so the HTTP request does not block on processing.
- Add drag-to-reorder for the Merge and Image-to-PDF tools instead of relying on upload order.

- Package the external binary dependencies into a single Docker image to remove the separate installation step from the deployment process.
- Add a lightweight test suite (e.g. `pytest`) that runs the same test cases listed in Table 5.1 automatically in a continuous-integration pipeline.

Bibliography

- [1] Flask Documentation. *Flask: A Python Micro Web Framework*. <https://flask.palletsprojects.com/>
- [2] React Documentation. *React – A JavaScript Library for Building User Interfaces*. <https://react.dev/>
- [3] pypdf Documentation. *pypdf: A Pure-Python PDF Library*. <https://pypdf.readthedocs.io/>
- [4] ReportLab Documentation. *ReportLab PDF Toolkit for Python*. <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- [5] Poppler Project. *Poppler: A PDF Rendering Library*. <https://poppler.freedesktop.org/>
- [6] Artifex Software. *Ghostscript Documentation*. <https://www.ghostscript.com/docs/>
- [7] The Document Foundation. *LibreOffice Headless Command-Line Conversion*. <https://help.libreoffice.org/>
- [8] Google / Tesseract OCR. *Tesseract Open Source OCR Engine*. <https://github.com/tesseract-ocr/tesseract>
- [9] iLovePDF. *Online PDF Tools for PDF Lovers*. <https://www.ilovepdf.com/>
- [10] Vite Documentation. *Vite: Next Generation Frontend Tooling*. <https://vitejs.dev/>

A. REST API Reference

Method	Endpoint	Request	Response
POST	/merge	files[] (2+ PDFs)	merged PDF
POST	/split	file, range	PDF of extracted pages
POST	/rotate	file, angle	rotated PDF
POST	/watermark	file, text	watermarked PDF
POST	/convert	file, format	image or ZIP of images
POST	/imagetopdf	files[] (1+ images)	PDF
POST	/compress	file	compressed PDF, with size headers
POST	/toword	file	.docx
POST	/wordtopdf	file (.docx)	PDF
POST	/edit/inspect	file	JSON page structure
POST	/edit/apply	file, edits (JSON)	edited PDF
GET	/health	—	{"status": "ok"}

Table A.1: Complete REST API endpoint reference

B. Sample Error Response

Every error response returned by the backend follows the same JSON shape, regardless of which endpoint or which validation rule triggered it:

```
1 {  
2   "error": true,  
3   "message": "Only PDF files are accepted."  
4 }
```

Listing B.1: Uniform error response shape